

DBT TRAINING

● Working Effectively • Module 08 • 45 min

Seeds and Variables

Version-controlled lookup tables and runtime-configurable constants — two tools that remove hardcoded values from your SQL.

The Problem: Hardcoded Lookups in SQL

Three models. Same lookup. Three places to update when a label changes.

mrt_deals_funnel.sql

```
SELECT
  deal_id,
  CASE country_code
    WHEN 'DE' THEN 'Germany'
    WHEN 'AT' THEN 'Austria'
    WHEN 'CH' THEN 'Switzerland'
    ELSE 'Other'
  END AS country_name
FROM fct_deal
```

mrt_contact_prescriptions.sql

```
SELECT
  contact_key,
  CASE country_code
    WHEN 'DE' THEN 'Germany'
    WHEN 'AT' THEN 'Austria'
    WHEN 'CH' THEN 'Switzerland'
    ELSE 'Other'
  END AS country_name
FROM dim_contact
```

mrt_country_summary.sql

```
SELECT
  CASE country_code
    WHEN 'DE' THEN 'Germany'
    WHEN 'AT' THEN 'Austria'
    WHEN 'CH' THEN 'Switzerland'
    ELSE 'Other'
  END AS country_name,
  COUNT(*) AS contacts
FROM dim_contact
GROUP BY 1
```

Switzerland expands DACH coverage to include Liechtenstein. **You need to add 'LI' to all three CASE statements.** Miss one → inconsistent country labels in Power BI.

Seeds: What They Are

A seed is a CSV file in your `seeds/` folder

```
dbt_project/
├── seeds/
│   ├── country_codes.csv
│   └── product_categories.csv
├── models/
└── ...
```

country_codes.csv :

```
country_code,country_name,region
DE,Germany,DACH
AT,Austria,DACH
CH,Switzerland,DACH
FR,France,Western Europe
```

How to load it

```
dbt seed
```

Creates a table in your warehouse from the CSV. Every model can reference it with `{{ ref('country_codes') }}`.

Why it works

The CSV lives in Git. Changes go through code review. You can `git diff` a label change. You can `git revert` a bad update.

The fix for slide 2

One row in `country_codes.csv`. All three models join it. Update once, correct everywhere.

Seeds: Two Patterns

Lookup seeds — production use

- Static reference tables: `country_codes` , `product_categories`
- Version-controlled in Git, curated by a human
- Available in all medallion layers via `{{ ref() }}`
- This is what the module teaches

Development seeds — local DuckDB only

- Files like `raw_contacts.csv` , `raw_deals.csv`
- Simulate the Bronze layer so exercises work without Snowflake
- In production, these are Lambda-ingested Bronze tables — not seeds
- Scaffolding only — this pattern does not go to production

Seeds: When to Use Them

Situation	Use...	Because...
Static lookup data you curate manually (country codes, status labels, category mappings)	seed	Version-controlled, small, changes only when you decide
Raw data loaded by an external pipeline (HubSpot contacts, deals, prescriptions)	source	A pipeline owns it — you observe it, you don't manage it
A dimension that derives from source data with transformation logic	dim table	dbt builds it from raw source data, not from a spreadsheet
Large reference data produced by a pipeline (e.g. product catalogue from ERP API)	source → staging → dim	Even "reference data" is a source if a pipeline populates it

The key question

Seeds Config in dbt_project.yml

Basic config

```
# dbt_project.yml
seeds:
  your_project:
    +schema: seeds
    +tags: ['reference']
    country_codes:
      +column_types:
        country_code: varchar(2)
        country_name: varchar(100)
        region: varchar(100)
```

+schema: seeds — loads to <target>_seeds schema instead of default

+column_types — prevents type inference errors (e.g. code 01 read as integer 1)

Useful commands

```
# Load all seeds
dbt seed

# Load one seed
dbt seed --select country_codes

# Force full reload
dbt seed --full-refresh
```

Column types matter

DuckDB and Snowflake infer types from CSV content. A country code of 01 becomes integer 1 without an explicit type override. Always define types for code columns.

Variables: The Problem They Solve

The same date constant in three models. Change the reporting window — edit three files.

fct_deal.sql

```
WHERE close_date  
      ≥ '2024-01-01'
```

fct_prescription.sql

```
WHERE prescribed_at  
      ≥ '2024-01-01'
```

mrt_deals_funnel.sql

```
WHERE deal_created_at  
      ≥ '2024-01-01'
```

The fix: one variable, one place to override

```
WHERE close_date ≥ '{{ var("start_date", "2024-01-01") }}'
```

Change the date across all models at runtime: `dbt run --vars '{"start_date": "2023-01-01"}'`

{{ var() }} Syntax

Basic syntax

```
-- No default - fails if var not set
WHERE close_date ≥ '{{ var("start_date") }}'
```



```
-- With default - safe without --vars
WHERE close_date ≥ '{{ var("start_date", "2024-01-01") }}
```

Always provide a default for variables used in production models. A missing `--vars` flag on a scheduled run breaks your pipeline silently.

Runtime override

```
# Single variable
dbt run --select fct_deal \
  --vars '{"start_date": "2023-01-01"}'
```



```
# Multiple variables
```

What the compiled SQL looks like

```
dbt compile --select fct_deal \
  --vars '{"start_date": "2023-06-01"}'
```



```
-- target/compiled/.../fct_deal.sql
SELECT deal_key, deal_id, amount, close_date
FROM dev.stg_hubspot_deals
WHERE close_date ≥ '2023-06-01'
--
      ↑ substituted
```

How dbt resolves the value

1. `--vars` flag (highest priority)
2. `dbt_project.yml` vars block
3. Default in `var("name", "default")`

Variables in dbt_project.yml

Defining project-wide defaults

```
# dbt_project.yml
name: 'dbt_training'
version: '1.0.0'

vars:
  start_date: '2024-01-01'
  min_deal_amount: 0
  env: 'dev'

models:
  dbt_training:
    +materialized: view
  silver:
    +materialized: table
```

All models in this project can call `{{ var("start_date") }}` without a default — the `dbt_project.yml` value is the fallback.

Pattern: environment guard

```
-- fct_deal.sql
SELECT deal_key, deal_id, amount
FROM {{ ref('stg_hubspot_deals') }}
{% if var('env', 'dev') = 'dev' %}
LIMIT 1000
{% endif %}
```

Speeds up dev runs. Production sets `env: prod` via CI.

Pattern: amount threshold

```
-- fct_deal.sql
WHERE amount ≥ {{ var('min_deal_amount', 0) }}
```

Zero in production (keep everything). Override in analysis:

```
-vars '{"min_deal_amount": 500}'
```

An error occurred on this slide. Check the terminal for more information.

Module 08 Summary

Seeds = version-controlled lookup tables

Put human-curated CSVs in `seeds/`, run `dbt seed`, reference with `{{ ref() }}`. One place to update, consistent everywhere.

Variables = runtime-configurable constants

Replace hardcoded dates and thresholds with `{{ var("name", "default") }}`.
Override at runtime with `--vars` without touching model files.

Both solve the same underlying problem

Hardcoded values scattered across multiple models are a maintenance trap.
Seeds extract lookup data. Variables extract constants. Same principle, different mechanism.

